Robert Rossi, Emily Chen, Ryan Vattuone

Analyzing Salary Fairness Across Different Companies and Locations

Abstract

Salaries vary widely across companies, locations, and occupations but a higher salary does not necessarily indicate higher purchasing power. In this project, we built a machine learning pipeline to analyze compensation fairness by combining job posting data, Bureau of Labor Statistics (BLS) occupational wage benchmarks, company information, and state-level Personal Consumption Expenditure data. The fairness index was constructed by comparing listed job salaries against the BLS average wage for the same state and job to label job compensation as underpaid, fair, or generous. Among the regression models evaluated, Elastic Net had the best performance with an R^2 of approximately 0.39. For clustering, DBSCAN performed much better than K-means with a silhouette score of 0.425 as compared to 0.144, suggesting that salary fairness data is better captured with density-based groupings as opposed to spherical clusters. Overall, this project shows how machine learning techniques can be applied to publicly available data to discover trends in compensation across the United States and potentially help people make more informed decisions about compensation packages.

Introduction

Motivation

Salaries are very different across different companies, locations, and occupations but a higher salary does not always mean a better purchasing power [1]. Different states are associated with different costs of living, expenses, and taxes, meaning that the same dollar could stretch further in some states as opposed to others. Many companies will adjust their compensation rates for similar roles across different states to account for these differences in state-dependent expenses. However, there is a lack of transparency on how companies adjust their compensation values from state to state, making it challenging for job seekers to reliably compare the real value compensation packages across states.

Objective

Our project will analyze salary data at different companies and determine whether employees are being generously, fairly, or under-compensated given their salary, occupation, and state. More specifically, we will analyze data on income by occupation and state (dataset 2), data on cost of living by state (dataset 3), as well as data on existing occupations, respective salaries, and location (dataset 1) to determine if salaries for occupations for certain companies are livable and fair.

Background & Related Work

Before starting on our data analysis, we looked into existing research on clustering and regression techniques to figure out how to best approach our analysis. We wanted to look into different regression models to use in our analysis. As covered in class, Linear Regression is a good baseline model to use. Building on it, we planned to also try Lasso and Ridge Regression to address potential overfitting. If we had time, Elastic Net was another option we aimed to experiment with [2]. As touched on in class, we knew that K Means would be a good first attempt at clustering but wanted to explore other algorithms that would better capture the similarities and differences in our dataset [3]. While K Means can be a good general approach to clustering, the result can vary greatly depending on the random initial centers selected. An algorithm that can be used to avoid this downside is Bisecting K Means. Bisecting K Means

uses a hierarchical approach to clustering where the initial cluster is split into two sub-clusters typically using sum of squared error (SSE) [4].

System Design & Implementation

Algorithm(s) Considered/Selected:

To analyze salary fairness across different companies and locations, we evaluated both supervised and unsupervised machine learning techniques. Since the fairness index is a continuous numerical value, regression algorithms were used to predict fairness scores, while clustering algorithms were used to identify natural groupings within the dataset.

1. Regression

For regression, we first implemented Linear Regression as a baseline model due to its simplicity and interpretability. We then evaluated Ridge Regression, Lasso Regression, and Elastic Net Regression. Ridge Regression applies L2 regularization to reduce the effects of multicollinearity, while Lasso Regression applies L1 regularization and can perform feature selection by shrinking less important coefficients toward zero. Elastic Net combines both L1 and L2 regularization and was evaluated to determine whether a combination of the two approaches could improve predictive performance.

2. PCA

We also explored dimensionality reduction techniques before regression. Principal Component Analysis was used to reduce the dimensionality of the feature space while preserving most of the variance in the data. Truncated Singular Value Decomposition (SVD) was also evaluated as an alternative dimensionality reduction method. Both PCA and SVD were combined with Elastic Net Regression to determine whether reducing the feature space could improve prediction accuracy and computational efficiency

3. K-Means and DBSCAN

For unsupervised learning, we evaluated K-Means clustering and DBSCAN. K-Means was selected because it is one of the most widely used clustering algorithms and efficiently partitions observations into clusters by minimizing within-cluster variance. We tested multiple values of k and evaluated cluster quality using metrics such as silhouette score, Davies-Bouldin index, Calinski-Harabasz score, and sum of squared error (SSE). We also explored DBSCAN, a density-based clustering algorithm that can discover clusters of arbitrary shape and identify noise points without requiring a predefined number of clusters. PCA was used as a preprocessing step before clustering to reduce dimensionality, remove noise, and improve computational efficiency. This allowed us to compare clustering performance both with and without dimensionality reduction and determine whether PCA improved cluster quality.

Technologies & Tools used (and why)

The project was implemented in Python using Jupyter Notebook as the primary development environment. Jupyter Notebook was selected because it provides an interactive environment that combines code, visualizations, and documentation, making it well suited for data analysis and machine learning experimentation.

Pandas and NumPy were used for data cleaning, preprocessing, feature engineering, dataset merging, and numerical computations. Scikit-learn served as the primary machine learning library and was used for train-test splitting, feature scaling, dimensionality reduction using PCA and Truncated SVD, regression

modeling, clustering, and model evaluation. Matplotlib was used to generate visualizations and compare algorithm performance across different metrics.

The project was organized into multiple notebooks corresponding to different stages of the workflow, including data cleaning, preprocessing, regression analysis, and clustering analysis. This modular structure improved readability, debugging, and experimentation.

Architecture-related Decisions (if applicable)
Several architecture-related decisions were made to improve the organization, maintainability, and reliability of the project. Rather than performing all processing steps in a single program, the workflow was divided into multiple stages consisting of data cleaning, dataset merging, feature engineering, preprocessing, regression analysis, and clustering analysis. Each stage was implemented in a separate notebook, allowing individual components to be tested and debugged independently.

A key design decision was to maintain separate datasets for modeling and analysis. After merging jobs, company, employment, and PCE datasets, two output datasets were generated. The first retained complete merged information for reference purposes, while the second contained only records with the salary, wage, and cost-of-living information required for modeling. This reduced unnecessary complexity during later stages of analysis.

Another important decision was to create a fairness index that compared a job's listed salary to the average salary reported by the Bureau of Labor Statistics for the same occupation and state. This fairness index served as the primary target variable for regression models and provided a standardized way to compare compensation across different locations and occupations.

To support machine learning algorithms, categorical variables were converted using one-hot encoding and frequently occurring skills were transformed into binary features. Separate preprocessing pipelines were maintained for regression and clustering. The regression pipeline included train-test splitting, missing-value imputation, and feature scaling to prevent data leakage. The clustering pipeline used the full dataset and applied scaling and dimensionality reduction prior to clustering since clustering is an unsupervised learning task.

Finally, intermediate datasets were exported between stages of the workflow. This modular pipeline reduced processing time during experimentation and allowed individual steps to be rerun without repeating the entire data preparation process.

Experiments / Proof of concept evaluation
Datasets Used
Dataset 1: https://www.kaggle.com/datasets/juanmerinobermejo/data-jobs-dataset [5]
Includes: company name, job title, salary information, location, and company attributes
Size: 100,000+ samples and 10-20 features before processing

Dataset 2: https://www.bls.gov/oes/tables.htm [6]
US Bureau of Labor Statistics Occupational Employment and Wage Statistics by State as of May 2023

Includes: area/location, occupation, total estimated number employed (excludes self-employed), mean hourly wage, mean annual wage
Size: 37,609 samples and 32 features before processing

Dataset 3:
https://apps.bea.gov/regional/downloadzip.htm?_gl=1*ej30jm*_ga*MTM4NzEwMzQ3NS4xNzc1OTM0Nzk2*_ga_J4698JNNFT*czE3NzU5MzQ3OTYkbzEkZzAkdDE3NzU5MzQ3OTYkajYwJGwwJGgw
[7]
Bureau of Economic Analysis Regional Economic Accounts Personal Consumption Expenditures (PCE) by State
Includes: state/location, region, description, unit, personal consumption expenditure
Size: 14,440 samples and 36 features before processing

Methodology (e.g., n-fold-cross validation, number of folds, size of training/test set etc.) (as applicable)

1. Cleaning the employment data
We started out by cleaning the employment dataset (employment.csv or also referred to as BLS data) to be used later as a wage benchmark for our fairness index calculation. Our program filtered the data to retain the state-level records and detailed occupation groups since these features were the most relevant for comparing jobs by location and occupation. Some of the wage values contained suppression codes like '*' and '#' which represented unavailable and unreliable data respectively. To handle this appropriately at the cleaning stage, we replaced these values with NaNs. Hourly wage columns were annualized by multiplying by 2080, which makes the assumption that these were full-time positions where employees worked 40 hours a week for 52 weeks in a year. Then, we specified the features to keep and gave them more descriptive names to make the code easier to understand. After that, we reformatted textual numbers (e.g. 123,456) by removing the commas and converting them into proper numerical data types. Then, any remaining invalid values were replaced with NaN. After this dataset was cleaned, it was then exported to be merged with the other datasets in later steps.

2. Cleaning the PCE data
We then began cleaning the PCE dataset by keeping only the total personal consumption expenditures and US states. Then, we renamed the columns to be more descriptive and exported the dataset for merging in later steps.

3. Cleaning the company data
Then, we proceeded to clean the company.csv dataset. This dataset included information like sector, revenue size, employee size, company score, and review count. Revenue and employee size were initially stored as categories like XXXS, XXS, XS, S, M, L, and more so we converted these categories into ranked numeric values. Sparse columns like director were removed since it had too many missing values to be of use in our subsequent analysis. Our program then cleaned company names by stripping extra spaces and standardizing capitalization. Since the same company name could appear multiple times, our program grouped the data by the company name and aggregated company information so that we would have one row per company. The dataset was then exported to be merged with other datasets in later steps.

4. Cleaning the jobs data

After cleaning the company dataset, we moved on to jobs. This was the main dataset of the project. We started by removing salary outliers by filtering out jobs that had mean salaries below $15,000 or above $1,000,000 since these annual salaries are less typical and would likely skew our models predictions. We also filtered the data to only retain US state abbreviations, filled missing values in columns like profile and remote status with default values ('Unspecified' and 'On-site' respectively). The skills column was stored as a string representation of a list so we converted it into an actual Python list and counted the number of skills for each job. To make merging easier later, we mapped each job group to a BLS occupation code. After that, we cleaned the company names in the same way as in the company.csv dataset for easier merging. Then, we selected and renamed the most relevant columns of data (e.g. job_title, company, state, annual salaries, etc.). Finally, the dataset was exported to be merged later.

5. Merging the datasets

After cleaning each dataset, we merged them into one large dataset. To do this, our program first merged the jobs dataset with the company dataset using the cleaned company name. Then, we merged the job + company data with the employment dataset using both the state and BLS occupation code. After that, the abbreviated state names were converted into the full state names and merged with the PCE dataset by state. We then exported two versions of the dataset. The first version contained the full merged dataset, merged.csv. The second version only kept rows that had salary, BLS wage, and PCE data as modelling.csv.

6. Fairness index and labels

To classify jobs as 'generous', 'fair', or 'underpaid', we created a fairness index as calculated with the equation $fairness\ index\ =\ \frac{annual\ mean}{BLS\ annual\ mean}$. This index compares the listed average salary for a job against the BLS average salary for the same occupation and state. Then, we assigned labels based on the values. If the fairness index was less than 0.9, the job was labeled as 'underpaid'. If it was between 0.9 and 1.1, the job was labeled as 'fair'. If the fairness index was over 1.1, it was labeled as 'generous'.

7. Preprocessing

In the preprocessing step, we removed columns that were not going to be useful for analysis like job titles, company names, city names, and more. These columns were more likely to create noise or negatively impact the performance of the model. For example, a column like company name would likely have thousands of unique entries which would create thousands of sparse features if encoded that could lead to overfitting. Retained categorical columns like job group and seniority were converted using one-hot encoding to turn every category into a binary column. For the skills column, we selected the top 30 skills from the list and created each into its own binary feature to preserve important skill information without creating too many columns.

8a. Splitting and scaling the data (Regression)

For regression analysis, we separated the dataset into features (X) and target (y). We set the fairness index as our target variable and removed irrelevant features like job ID and the fairness index. Splitting the data is important in preventing data leakage since the model should not be trained on the values that it will be tested on. For this program, we went with an 80/20 split. At this point, some columns still had missing values like total employment and BLS 90th percentile wages. Our program filled missing values with the

median from the relevant training set values so as to avoid data leakage. Next, we scaled the numeric columns using StandardScaler. This standardized the values in the numeric columns so that no one column would have an outsized impact on the output of the model. The scaler was only fit on the training data and then applied to both the training and test data to, again, avoid data leakage.

8b. Scaling the data (Clustering)

We created a separate scaled feature set for clustering where columns like fairness index and fairness label were removed. Since, unlike regression (which is a supervised learning task), clustering is unsupervised, we did not split the dataset used for clustering. We scaled the data in a similar manner as in Step 8a using StandardScaler.

Graphs

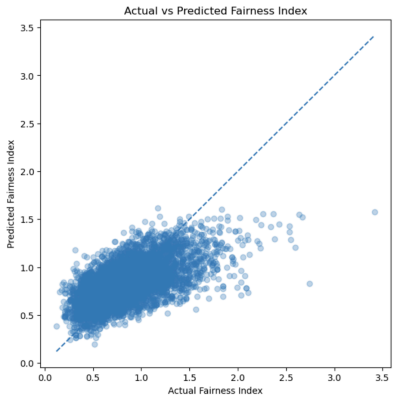


Figure 1. Actual versus predicted fairness index values using the Elastic Net regression model.

Figure 1 shows a moderate positive relationship between the predicted and actual fairness index values. While the Elastic Net model captures general trends in compensation fairness, prediction error remains for some observations. This result is consistent with the model's R^2 value of approximately 0.39.

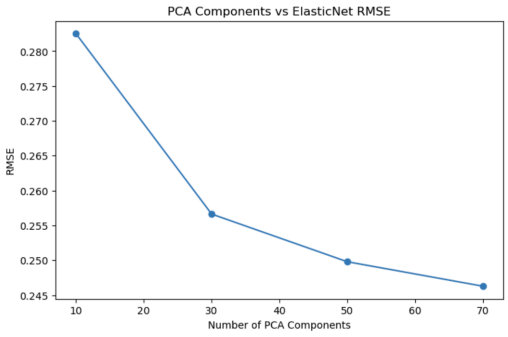


Figure 2. Effect of the number of retained PCA components on Elastic Net regression performance measured by RMSE.

As the number of retained PCA components increases, RMSE decreases and model performance improves. However, even the best PCA configuration does not outperform the original feature set, indicating that dimensionality reduction results in some loss of predictive information.

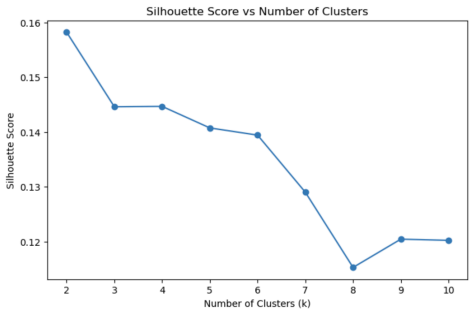


Figure 3. Silhouette scores for K-Means clustering across different values of k.

The silhouette score was highest when k = 2 and generally decreased as the number of clusters increased. This suggests that the salary fairness dataset naturally forms a small number of broad groups rather than many highly separated clusters.

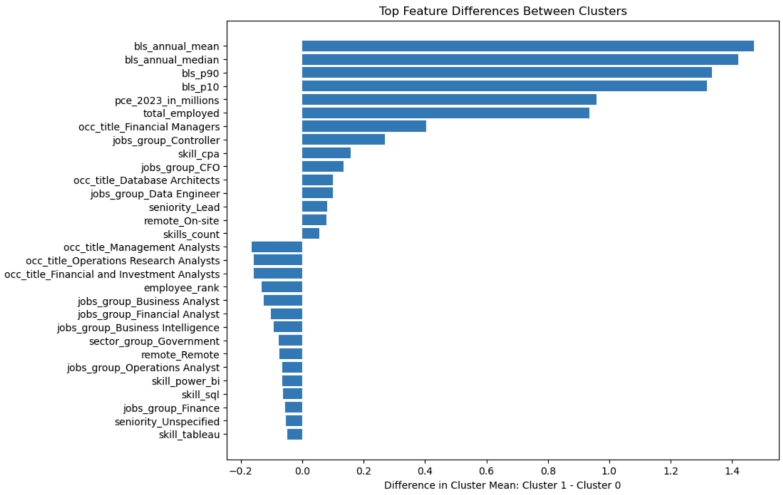


Figure 4. DBSCAN clustering results visualized using the first two principal components.

The DBSCAN clustering results show the presence of distinct groups within the dataset while also identifying potential noise points. Compared to K-Means, DBSCAN produced stronger clustering metrics, suggesting that density-based clustering better captures the structure of the salary fairness data.

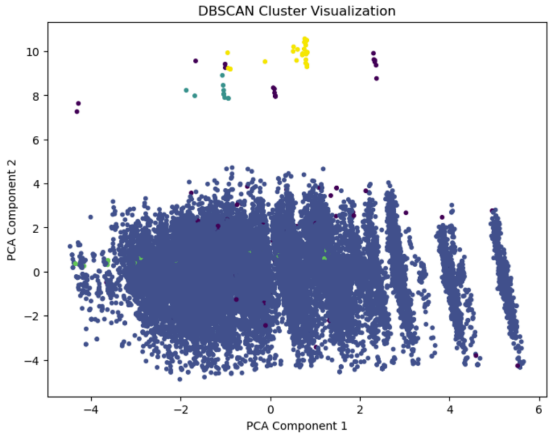


Figure 5. Features exhibiting the largest differences between K-Means cluster centroids.
The largest differences between cluster centroids were associated with wage, employment, and occupation-related features. These differences help explain how K-Means separated observations into distinct groups and provide insight into the characteristics that drive cluster formation.

Analysis of results

1. Regression Results

The regression experiments showed that linear models were capable of predicting the fairness index with moderate accuracy. Linear Regression achieved an RMSE of approximately 0.2455 and an R^2 value of approximately 0.3897. Ridge Regression, Lasso Regression, and Elastic Net Regression were then evaluated to determine whether regularization could improve predictive performance. While all three regularized models performed similarly, Elastic Net achieved the best overall results with an RMSE of approximately 0.2455 and an R^2 value of approximately 0.3898. The small performance differences between the models suggests that the dataset does not suffer from severe overfitting or multicollinearity and that the engineered feature set was relatively stable.

2. PCA and SVD Results

We also evaluated dimensionality reduction techniques before regression. PCA and Truncated SVD were combined with Elastic Net Regression using different numbers of components. As the number of retained components increased, model performance improved. However, neither PCA nor SVD surpassed the performance of the original feature set. The best PCA configuration achieved an R^2 of approximately 0.3860 using 70 components, while the best SVD configuration achieved an R^2 of approximately 0.3835. These results indicate that although dimensionality reduction reduced the number of features, some predictive information was lost in the process.

3. Clustering

For clustering, K-Means was evaluated for values of k ranging from 2 to 10 using silhouette score, Davies-Bouldin index, Calinski-Harabasz score, and SSE. As expected, SSE decreased as the number of clusters increased, indicating that the clusters became more compact. The highest silhouette score was obtained when $k = 2$, suggesting that the data naturally forms only a small number of broad groups rather than many highly separated clusters. We also evaluated the impact of PCA on clustering performance. Applying PCA before K-Means improved the silhouette score from approximately 0.144 to 0.158 while also improving the Davies-Bouldin and Calinski-Harabasz scores. This suggests that PCA successfully removed noise and redundant information, resulting in better cluster separation. Finally, DBSCAN was evaluated as a density-based clustering alternative. DBSCAN achieved a silhouette score of approximately 0.425 and a Davies-Bouldin score of approximately 0.802, substantially outperforming K-Means on these clustering quality metrics. These results suggest that the underlying structure of the salary fairness data is not well represented by spherical clusters and that density-based clustering is better able to capture the natural groupings present in the dataset.

Overall, the results indicate that salary fairness can be partially predicted using publicly available information about occupations, companies, skills, and locations. While the regression models explained roughly 39% of the variation in the fairness index, the clustering results revealed meaningful groupings within the data, particularly when dimensionality reduction and density-based clustering techniques were applied.

Discussion & Conclusions

Decisions made

Several important decisions were made throughout the project to improve the quality and interpretability of the analysis. One of the most significant decisions was creating a fairness index that compared a job's listed salary to the average salary reported by the Bureau of Labor Statistics (BLS) for the same occupation and state. This provided a standardized metric that could be used to compare compensation across different locations and occupations. We also removed salary outliers below $15,000 and above $1,000,000 to reduce the influence of extreme values and focused only on occupations with available benchmark wage data.

During preprocessing, categorical variables were encoded using one-hot encoding, while the most common skills were converted into binary features to preserve important information without creating excessive sparsity. For regression analysis, we selected an 80/20 train-test split and applied scaling using only the training data to prevent data leakage. For clustering, PCA was applied before evaluating both K-Means and DBSCAN to reduce dimensionality and compare different clustering approaches.

Difficulties faced/Things that didn't work well

The main difficulty during cleaning and preprocessing was mapping real-world job titles to the standardized occupation codes used by the Bureau of Labor Statistics (BLS). Job titles often varied significantly while corresponding to the same occupation code, and the limited granularity of BLS classifications sometimes grouped positions with different responsibilities and salaries into the same category. To address this issue, we manually mapped job groups to BLS occupation codes, since misclassifications could significantly affect the fairness index and subsequent analysis. Ultimately, 93.4% of job titles were successfully mapped, though some remained too ambiguous to assign a reliable wage benchmark. The high dimensionality of the dataset also presented challenges. One-hot encoding categorical variables and extracting skill-related features substantially increased the number of features, creating the potential for overfitting and increased computational complexity. To address this issue, dimensionality reduction techniques such as PCA and Truncated SVD were evaluated. Additionally, selecting appropriate machine learning models and hyperparameters required significant experimentation, particularly when determining the number of clusters for K-Means and tuning the density-based parameters used by DBSCAN.

Things that worked

Several decisions during the cleaning and preprocessing stages proved effective. Replacing BLS suppression codes (* and #) with NaNs preserved data that would otherwise have been lost while still handling missing values appropriately. Converting revenue and employee size categories into ranked numeric values retained useful information without creating sparse features through one-hot encoding. Removing salary outliers below $15,000 and above $1,000,000 also helped reduce the influence of extreme values while requiring only a small number of rows to be excluded. Dimensionality reduction using PCA was also successful for clustering, improving clustering quality metrics while reducing the dimensionality of the feature space. Additionally, DBSCAN outperformed K-Means on several clustering metrics, suggesting that density-based clustering was better suited for capturing the underlying structure of the salary fairness data.

Conclusion

In this project, we developed a machine learning pipeline to analyze salary fairness across different companies and geographic locations. By combining job posting data, company information, Bureau of Labor Statistics wage data, and state-level personal consumption expenditure data, we created a fairness index that measures compensation relative to occupation and location benchmarks. The resulting dataset was cleaned, merged, preprocessed, and analyzed using both regression and clustering techniques.

Our results showed that salary fairness can be partially predicted using publicly available information about occupations, companies, skills, and locations. Among the regression models evaluated, Elastic Net achieved the best predictive performance, although the improvement over the other linear models was relatively small. We also found that PCA improved clustering quality and that DBSCAN produced stronger clustering metrics than K-Means, suggesting that density-based clustering better captures the structure of the data. Overall, this project demonstrated how machine learning and publicly available economic data can be combined to evaluate compensation fairness and identify patterns in salary data. Future work could incorporate additional factors such as years of experience, education level, company size, and job performance metrics to improve predictive accuracy and provide a more comprehensive assessment of salary fairness.

References

[1] "Nominal Wage vs. Real Wage: Definitions and Examples," *Indeed Career Guide*, 2022. https://www.indeed.com/career-advice/career-development/nominal-wage-vs-real-wage

[2] A. Ng, "CS229 Lecture notes Supervised learning." Available: https://sgfin.github.io/files/notes/CS229_Lecture_Notes.pdf

[3] A. Ng, "CS229 Lecture notes The k-means clustering algorithm." Accessed: Apr. 06, 2024. [Online]. Available: https://cs229.stanford.edu/notes2020spring/cs229-notes7a.pdf [kmeans reference]

[4] V. Rohilla, S. Singh, S. Chakraborty, and M. S. Singh, "Data Clustering using Bisecting K-Means," *2022 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Oct. 2019, doi: https://doi.org/10.1109/icccis48478.2019.8974537

[5] J. Merino, "US Data Jobs Salaries Dataset," *Kaggle.com*, 2023. https://www.kaggle.com/datasets/juanmerinobermejo/data-jobs-dataset (accessed May 15, 2026).

[6] U.S. Bureau of Labor Statistics, "Occupational Employment and Wage Statistics," *Bls.gov*, Aug. 26, 2019. https://www.bls.gov/oes/tables.htm

[7] "Consumer Spending by State | U.S. Bureau of Economic Analysis (BEA)," *www.bea.gov*. https://www.bea.gov/data/consumer-spending/state